KUBERNETES CON LOAD BALANCER Y AWS

Héctor Ávila Manjón
2ºASIR

PARTE 0: PREPARACIÓN DEL ENTORNO LOCAL EN WSL2

0.1 – Instalar k3s

sudo apt update – y && sudo apt upgrade -y

```
hect@A6Alumno13:~$ sudo apt update -y && sudo apt upgrade
[sudo] password for hect:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1404 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [228 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9724 B]
```

sudo apt install -y curl wget git

```
hect@A6Alumno13:~$ sudo apt install -y curl wget git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (8.5.0-2ubuntu10.6).
curl set to manually installed.
```

curl -sfL https://get.k3s.io | K3S_KUBECONFIG_MODE="644" sh -

```
hect@A6Alumno13:~$ curl -sfL https://get.k3s.io | K3S_KUBECONFIG_MODE="644" sh -
[INFO]  Finding release for channel stable
[INFO]  Using v1.34.3+k3s1 as release
[INFO]  Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/sha256sum-amd64.txt
[INFO]  Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/k3s
[INFO]  Verifying binary download
[INFO]  Installing k3s to /usr/local/bin/k3s
[INFO]  Skipping installation of SELinux RPM
[INFO]  Creating /usr/local/bin/kubectl symlink to k3s
```

sudo k3s --version

```
hect@A6Alumno13:~$ sudo k3s --version
k3s version v1.34.3+k3s1 (48ffa7b6)
go version go1.24.11
```

sudo k3s server & sleep 10

```
hect@A6Alumno13:~$ sudo k3s server & sleep 10
[1] 6612
INFO[0000] Starting k3s v1.34.3+k3s1 (48ffa7b6)
INFO[0000] Configuring sqlite3 database connection pooling:
maxIdleConns=2, maxOpenConns=0, connMaxLifetime=0s
INFO[0000] Configuring database table schema and indexes, this may
take a moment...
INFO[0000] Database tables and indexes are up to date
INFO[0000] Kine available at unix://kine.sock
INFO[0000] Reconciling bootstrap data between datastore and disk
```

sudo k3s kubectl get nodes

```
hect@A6Alumno13:~$ sudo k3s kubectl get nodes
NAME           STATUS   ROLES                  AGE    VERSION
a6alumno14     Ready    control-plane          45s    v1.34.3+k3s1
```

0.2 – Instalar kubectl localmente (para comodidad)

curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo chmod +x kubectl
sudo mv kubectl /usr/local/bin/

```
hect@A6Alumno13:~$ curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo chmod +x kubectl
sudo mv kubectl /usr/local/bin/
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   138  100   138    0     0    627      0 --:--:-- --:--:-- --:--:--   630
100 55.8M  100 55.8M    0     0  44.8M      0  0:00:01  0:00:01 --:--:-- 70.1M
```

kubectl version --client

```
hect@A6Alumno13:~$ kubectl version --client
Client Version: v1.35.0
Kustomize Version: v5.7.1
```

mkdir -p $HOME/.kube
sudo cp /etc/rancher/k3s/k3s.yaml $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
sudo chmod 600 $HOME/.kube/config

```
hect@A6Alumno13:~$ mkdir -p $HOME/.kube
sudo cp /etc/rancher/k3s/k3s.yaml $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
sudo chmod 600 $HOME/.kube/config
```

kubectl get nodes

```
hect@A6Alumno13:~$ kubectl get nodes
NAME           STATUS   ROLES                  AGE    VERSION
a6alumno14     Ready    control-plane          107s   v1.34.3+k3s1
```

mkdir -p ~/kubernetes-aws-practice
cd ~/kubernetes-aws-practice

```
hect@A6Alumno13:~$ mkdir -p ~/kubernetes-aws-practice
cd ~/kubernetes-aws-practice
hect@A6Alumno13:~/kubernetes-aws-practice$
```

PARTE 1: CREAR APLICACIÓN SIMPLE PARA KUBERNETES

1.1 – Crear carpeta para la aplicación

mkdir -p ~/kubernetes-aws-practice/app

cd ~/kubernetes-aws-practice/app

```
hect@A6Alumno13:~/kubernetes-aws-practice/app$ mkdir -p ~/kubernetes-aws-practice/app
cd ~/kubernetes-aws-practice/app
```

1.2 – Crear aplicación Python con Flask

cat > app.py << 'EOF'

```
hect@A6Alumno13:~/kubernetes-aws-practice/app$ cat > app.py << 'EOF'
#!/usr/bin/env python3
from flask import Flask, jsonify, send_from_directory
import os
import socket
from datetime import datetime
import sys

app = Flask(__name__)

# Variables de entorno inyectadas por Kubernetes
POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')
```

#!/usr/bin/env python3
from flask import Flask, jsonify, send_from_directory
import os
import socket
from datetime import datetime
import sys
app = Flask(__name__)
# Variables de entorno inyectadas por Kubernetes
POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')
@app.route('/')
def index():
return send_from_directory('.', 'index.html')
@app.route('/pod-info')
def pod_info():
return jsonify({
'pod_name': POD_NAME,
'namespace': POD_NAMESPACE,
'hostname': socket.gethostname(),
'timestamp': datetime.now().isoformat()
})
@app.route('/health')
def health():
return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200

if __name__ == '__main__':
print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.stderr)
app.run(host='0.0.0.0', port=5000, debug=False)
EOF
# Dar permisos de ejecución (Importante)
sudo chmod +x app.py

1.3 – Crear archivo HTML

cat > index.html << 'EOF'

```
hect@A6Alumno13:~/kubernetes-aws-practice/app$ cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
<head>
    <title>Kubernetes Load Balancer</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
            margin: 0;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
```

cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
<head>
<title>Kubernetes Load Balancer</title>
<style>
body {
font-family: Arial, sans-serif;
display: flex;
justify-content: center;
align-items: center;
min-height: 100vh;
margin: 0;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}
.container {
background: white;
padding: 50px;
border-radius: 10px;
box-shadow: 0 10px 25px rgba(0,0,0,0.2);
text-align: center;
max-width: 500px;
}
h1 {
color: #667eea;

```
margin: 0 0 30px 0;
}
.info {
background: #f0f0f0;
padding: 20px;
border-radius: 5px;
margin: 20px 0;
}
.pod-name {
font-size: 28px;
color: #764ba2;
font-weight: bold;
font-family: monospace;
}
.timestamp {
color: #666;
font-size: 13px;
margin-top: 10px;
}
.instruction {
background: #e0f2fe;
padding: 15px;
border-radius: 5px;
color: #0369a1;
font-size: 14px;
margin-top: 20px;
}
.refresh-button {
margin-top: 20px;
padding: 10px 20px;
background: #667eea;
color: white;
border: none;
border-radius: 5px;
cursor: pointer;
font-size: 14px;
}
.refresh-button:hover {
background: #764ba2;
}
</style>
</head>
<body>
<div class="container">
<h1>Kubernetes Load Balancer</h1>
<div class="info">
<p style="margin: 0 0 10px 0; color: #666;">Pod atendiendo solicitud:</p>
<p class="pod-name" id="pod-name">Cargando...</p>
```

* *
* *
* *
Actualiza constantemente esta página para ver el balanceo de carga en acción
* *
<button class="refresh-button" onclick="location.reload()">Actualizar Ahora</button>
* *
<script>
fetch('/pod-info')
.then(r => r.json())
.then(data => {
document.getElementById('pod-name').textContent = data.pod_name;
const date = new Date(data.timestamp);
document.getElementById('timestamp').textContent = date.toLocaleString('es-ES');
})
.catch(e => {
document.getElementById('pod-name').textContent = 'Error';
console.error('Error:', e);
});
</script>
</body>
</html>
EOF

1.4 – Crear requirements.txt

"cat > requirements.txt << 'EOF'
Flask==3.0.0
Werkzeug==3.0.0
EOF".

```
hect@A6Alumno13:~/kubernetes-aws-practice/app$ cat > requirements.txt << 'EOF'
Flask==3.0.0
Werkzeug==3.0.0
EOF
```

1.5 – Crear Dockerfile ultraligero

cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
 # Copiar archivos
COPY requirements.txt .
COPY app.py .
COPY index.html .
 # Instalar dependencias
RUN pip install --no-cache-dir -r requirements.txt
 # Ejecutar app
CMD ["python", "app.py"]
EOF

```
hect@A6Alumno13:~/kubernetes-aws-practice/app$ cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
# Copiar archivos
COPY requirements.txt .
COPY app.py .
COPY index.html .
# Instalar dependencias
RUN pip install --no-cache-dir -r requirements.txt
# Ejecutar app
CMD ["python", "app.py"]
EOF
```

PARTE 2: CONFIGURAR KUBERNETES (MANIFESTS YAML)

2.1 – Crear Namespace

cd ~/kubernetes-aws-practice

"cat > namespace.yaml << 'EOF'
apiVersion: v1
kind: Namespace
metadata:
name: load-balancer-demo
labels:
name: load-balancer-demo
EOF".

```
hect@A6Alumno13:~/kubernetes-aws-practice$ cat > namespace.yaml << 'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: load-balancer-demo
  labels:
    name: load-balancer-demo
EOF
```

kubectl apply -f namespace.yaml
kubectl get namespaces

```
hect@A6Alumno13:~/kubernetes-aws-practice$ kubectl apply -f namespace.yaml
kubectl get namespaces
namespace/load-balancer-demo created
NAME                 STATUS   AGE
default              Active   66m
kube-node-lease      Active   66m
kube-public          Active   66m
kube-system          Active   66m
load-balancer-demo   Active   0s
```

2.2 – Crear ConfigMap con archivos de la app

```
cat > configmap.yaml << 'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
name: app-files
namespace: load-balancer-demo
data:
requirements.txt: |
Flask==3.0.0
Werkzeug==3.0.0
app.py: |
#!/usr/bin/env python3
from flask import Flask, jsonify, send_from_directory
import os
import socket
from datetime import datetime
import sys
app = Flask(__name__)
POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')
@app.route('/')
def index():
return send_from_directory('.', 'index.html')
@app.route('/pod-info')
def pod_info():
return jsonify({
'pod_name': POD_NAME,
'namespace': POD_NAMESPACE,
'hostname': socket.gethostname(),
'timestamp': datetime.now().isoformat()
})
@app.route('/health')
def health():
return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200
if __name__ == '__main__':
print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.stderr)
app.run(host='0.0.0.0', port=5000, debug=False)
index.html: |
<!DOCTYPE html>
<html>
<head>
<title>Kubernetes Load Balancer</title>
<style>
body { font-family: Arial, sans-serif; display: flex; justify-content: center; align-items:
center; min-height: 100vh; margin: 0; background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%); }
.container { background: white; padding: 50px; border-radius: 10px; box-shadow: 0
```

10px 25px rgba(0,0,0,0.2); text-align: center; max-width: 500px; }
h1 { color: #667eea; margin: 0 0 30px 0; }
.info { background: #f0f0f0; padding: 20px; border-radius: 5px; margin: 20px 0; }
.pod-name { font-size: 28px; color: #764ba2; font-weight: bold; font-family: monospace; }
.instruction { background: #e0f2fe; padding: 15px; border-radius: 5px; color: #0369a1; font-size: 14px; margin-top: 20px; }
.refresh-button { margin-top: 20px; padding: 10px 20px; background: #667eea; color: white; border: none; border-radius: 5px; cursor: pointer; }
</style>
</head>
<body>
* *
<h1>Kubernetes Load Balancer</h1>
* *
* Pod atendiendo solicitud: *
* Cargando... *
* *
* *
* Actualiza para ver el balanceo de carga *
<button class="refresh-button" onclick="location.reload()">Actualizar Ahora</button>
* *
<script>
fetch('/pod-info').then(r => r.json()).then(data => {
document.getElementById('pod-name').textContent = data.pod_name;
document.getElementById('timestamp').textContent = new Date(data.timestamp).toLocaleString('es-ES');
});
</script>
</body>
</html>
EOF

```
hect@A6Alumno13:~/kubernetes-aws-practice$ cat > configmap.yaml << 'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-files
  namespace: load-balancer-demo
data:
  requirements.txt: |
    Flask==3.0.0
    Werkzeug==3.0.0
  app.py: |
```

kubectl apply -f configmap.yaml
kubectl get configmap -n load-balancer-demo

```
hect@A6Alumno13:~/kubernetes-aws-practice$ kubectl apply -f configmap.yaml; kubectl get configmap -n
load-balancer-demo
configmap/app-files created
NAME        DATA   AGE
app-files   3      1s
```

2.3 – Crear Deployment con 3 replicas

cat > deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
name: web-app
namespace: load-balancer-demo
labels:
app: web-app
spec:
replicas: 3
selector:
matchLabels:
app: web-app
template:
metadata:
labels:
app: web-app
spec:
containers:
- name: web-app
image: python:3.11-slim
command: ["sh", "-c"]
args:
- |
cd /app
pip install --no-cache-dir -r requirements.txt > /dev/null 2>&1
python app.py
ports:
- containerPort: 5000
env:
- name: POD_NAME
valueFrom:
fieldRef:
fieldPath: metadata.name
- name: POD_NAMESPACE
valueFrom:
fieldRef:
fieldPath: metadata.namespace
volumeMounts:

- name: app-volume
mountPath: /app
livenessProbe:
httpGet:
path: /health
port: 5000
initialDelaySeconds: 15
periodSeconds: 10
readinessProbe:
httpGet:
path: /health
port: 5000
initialDelaySeconds: 5
periodSeconds: 5
volumes:
- name: app-volume
configMap:
name: app-files
defaultMode: 0755
EOF

```
hect@A6Alumno13:~/kubernetes-aws-practice$ cat > deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: load-balancer-demo
  labels:
    app: web-app
spec:
  replicas: 3
```

kubectl apply -f deployment.yaml
kubectl wait --for=condition=ready pod -l app=web-app -n load-balancer-demo --timeout=120s

```
hect@A6Alumno13:~/kubernetes-aws-practice$ kubectl wait --for=condition=ready pod -l app=web-app -n
load-balancer-demo --timeout=120s
pod/web-app-65967466dd-2m4bg condition met
pod/web-app-65967466dd-5wwqc condition met
pod/web-app-65967466dd-8c6ws condition met
```

2.4 – Crear Service (Load Balancer)

cat > service.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
name: web-app-service
namespace: load-balancer-demo
spec:
type: LoadBalancer
selector:
app: web-app
ports:
- protocol: TCP
port: 80
target_port: 5000
EOF

```
hect@A6Alumno13:~/kubernetes-aws-practice$ cat > service.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: web-app-service
  namespace: load-balancer-demo
spec:
  type: LoadBalancer
  selector:
    app: web-app
  ports:
```

kubectl apply -f service.yaml
kubectl get svc -n load-balancer-demo

```
hect@A6Alumno13:~/kubernetes-aws-practice$ kubectl apply -f service.yaml
kubectl get svc -n load-balancer-demo
service/web-app-service created
NAME              TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
web-app-service   LoadBalancer   10.43.94.0     <pending>     80:32307/TCP   0s
```

PARTE 3: VERIFICAR BALANCEO DE CARGA LOCAL

3.1 – Levantar la aplicación

kubectl port-forward -n load-balancer-demo svc/web-app-service 8080:80

```
hect@A6Alumno13:~/kubernetes-aws-practice$ kubectl port-forward -n load-balancer-demo svc/web-
app-service 8080:80
Forwarding from 127.0.0.1:8080 -> 5000
Forwarding from [::1]:8080 -> 5000
```

3.2 – Probar balanceo con curl

“for i in {1..10}; do
echo "Petición $i:"
curl -s http://localhost:8080/pod-info | python3 -m json.tool | grep pod_name
sleep 1
done”.

```
hect@A6Alumno13:~$ for i in {1..10}; do
  echo "Petición $i:"
  curl -s http://localhost:8080/pod-info | python3 -m json.tool | grep pod_name
  sleep 1
  done
Petición 1:
    "pod_name": "web-app-65967466dd-2m4bg",
Petición 2:
    "pod_name": "web-app-65967466dd-2m4bg",
Petición 3:
    "pod_name": "web-app-65967466dd-2m4bg",
```

3.3 – Verificar en navegador

http://localhost:8080 en el navegador

PARTE 4: CONFIGURACIÓN EN AWS

4.1 – Crear Security Group

Agregar reglas de entrada:

Tipo	Protocolo	Puerto	Origen
SSH	**TCP**	22	0.0.0.0/0
HTTP	**TCP**	80	0.0.0.0/0
HTTPS	**TCP**	443	0.0.0.0/0

4.2 – Crear instancia EC2

1. **EC2** → Instances → Launch instances
2. **Nombre:** kubernetes-test-server
3. **AMI:** Ubuntu 24.04 LTS
4. **Tipo:** t3.micro (*Free Tier*)
5. **Key pair:** Tu clave SSH
6. **VPC:** Default
7. **Security group:** kubernetes-aws-sg
8. **Storage:** 8 GiB, gp3
9. Launch instance

4.3 – Conectar a EC2

ssh -i ~/.ssh/tu-clave-aws.pem ubuntu@IP

```
hect@A6Alumno13:~$ ssh -i ~/.ssh/kubernetes.pem ubuntu@54.167.4.190
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:     https://landscape.canonical.com
* Support:        https://ubuntu.com/pro
```

sudo apt update

sudo apt install -y curl wget python3 python3-pip git

```
ubuntu@ip-172-31-22-222:~$ sudo apt update
sudo apt install -y curl wget python3 python3-pip git
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
```

mkdir -p ~/kubernetes-test

```
ubuntu@ip-172-31-22-222:~$ mkdir -p ~/kubernetes-test
cd ~/kubernetes-test
```

PARTE 5: CONECTAR KUBERNETES LOCAL CON AWS EC2

5.1 – Crear túnel SSH que expone el LoadBalancer

En una nueva máquina local

*ssh -i ~/.ssh/tu-clave-aws.pem *
*-N -R 8888:localhost:8080 *
ubuntu@54.167.4.190

```
hect@A6Alumno13:~$ ssh -i ~/.ssh/kubernetes.pem -N -R 8888:localhost:8080 ubuntu@54.167.4.190
```

5.2 – Crear script de prueba en EC2

cat > ~/test-kubernetes-lb.sh << 'EOF'
#!/bin/bash
echo "=== Prueba Kubernetes Load Balancer desde AWS ==="
echo ""
declare -A pods_count
for i in {1..15}; do
 # Pedimos información al túnel local en el puerto 8888
response=$(curl -s http://localhost:8888/pod-info)
pod_name=$(echo $response | python3 -c "import sys, json;
print(json.load(sys.stdin)['pod_name'])" 2>/dev/null)
if [-z "$pod_name"]; then
pod_name="ERROR"
fi
echo "Petición $i → Pod: $pod_name"
pods_count[$pod_name]=$((${pods_count[$pod_name]:-0} + 1))
sleep 1
done
echo ""
echo "=== Resumen Balanceo ==="
for pod in "${!pods_count[@]}"; do
echo "Pod $pod: ${pods_count[$pod]} peticiones"
done
EOF

```
ubuntu@ip-172-31-22-222:~/kubernetes-test$ cat > ~/test-kubernetes-lb.sh << 'EOF'
#!/bin/bash
echo "=== Prueba Kubernetes Load Balancer desde AWS ==="
echo ""
declare -A pods_count
for i in {1..15}; do
  response=$(curl -s http://localhost:8888/pod-info)
```

sudo chmod +x ~/test-kubernetes-lb.sh
~/test-kubernetes-lb.sh

```
ubuntu@ip-172-31-22-222:~/kubernetes-test$ chmod +x ~/test-kubernetes-lb.sh
test-kubernetes-lb.sh
= Prueba Kubernetes Load Balancer desde AWS ===

tición 1 → Pod: web-app-65967466dd-2m4bg
tición 2 → Pod: web-app-65967466dd-2m4bg
tición 3 → Pod: web-app-65967466dd-2m4bg
tición 4 → Pod: web-app-65967466dd-2m4bg
tición 5 → Pod: web-app-65967466dd-2m4bg
tición 6 → Pod: web-app-65967466dd-2m4bg
tición 7 → Pod: web-app-65967466dd-2m4bg
tición 8 → Pod: web-app-65967466dd-2m4bg
tición 9 → Pod: web-app-65967466dd-2m4bg
tición 10 → Pod: web-app-65967466dd-2m4bg
tición 11 → Pod: web-app-65967466dd-2m4bg
tición 12 → Pod: web-app-65967466dd-2m4bg
tición 13 → Pod: web-app-65967466dd-2m4bg
tición 14 → Pod: web-app-65967466dd-2m4bg
tición 15 → Pod: web-app-65967466dd-2m4bg

= Resumen Balanceo ===
d web-app-65967466dd-2m4bg: 15 peticiones
```

PARTE 6: ESCALADO DINÁMICO

6.1 – Escalar a 5 replicas

En tu máquina local:

kubectl scale deployment web-app -n load-balancer-demo --replicas=5

```
hect@A6Alumno13:~$ kubectl scale deployment web-app -n load-balancer-demo --replicas=5
deployment.apps/web-app scaled
hect@A6Alumno13:~$
```

kubectl get pods -n load-balancer-demo

```
hect@A6Alumno13:~$ kubectl get pods -n load-balancer-demo
NAME                       READY   STATUS    RESTARTS        AGE
web-app-65967466dd-2m4bg   1/1     Running   1 (136m ago)    138m
web-app-65967466dd-5wwqc   1/1     Running   1 (136m ago)    138m
web-app-65967466dd-8c6ws   1/1     Running   1 (136m ago)    138m
web-app-65967466dd-fjb6s   1/1     Running   0               13s
web-app-65967466dd-tvgvb   1/1     Running   0               13s
```

kubectl wait --for=condition=ready pod -l app=web-app -n load-balancer-demo --timeout=120s

```
hect@A6Alumno13:~$ kubectl wait --for=condition=ready pod -l app=web-app -n load-balancer-demo --timeout=120s
pod/web-app-65967466dd-2m4bg condition met
pod/web-app-65967466dd-5wwqc condition met
pod/web-app-65967466dd-8c6ws condition met
pod/web-app-65967466dd-fjb6s condition met
pod/web-app-65967466dd-tvgvb condition met
```

6.2 – Probar balanceo desde AWS

En EC2:

~/test-kubernetes-lb.sh

```
ubuntu@ip-172-31-22-222:~/kubernetes-test$ ~/test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===

Petición 1 → Pod: web-app-65967466dd-2m4bg
```

PARTE 7: MONITOREO Y OBSERVABILIDAD

7.1 – Ver estado de pods

kubectl get pods -n load-balancer-demo

```
hect@A6Alumno13:~$ kubectl get pods -n load-balancer-demo
NAME                       READY   STATUS    RESTARTS        AGE
web-app-65967466dd-2m4bg   1/1     Running   1 (152m ago)    154m
web-app-65967466dd-5wwqc   1/1     Running   1 (152m ago)    154m
web-app-65967466dd-8c6ws   1/1     Running   1 (152m ago)    154m
web-app-65967466dd-fjb6s   1/1     Running   0               16m
web-app-65967466dd-tvgvb   1/1     Running   0               16m
```

kubectl logs -n load-balancer-demo web-app-xxxxx-11111

```
10.42.0.1 - - [16/Jan/2026 12:37:49] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:37:54] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:37:56] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:37:59] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:04] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:06] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:10] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:15] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:17] "GET /health HTTP/1.1" 200 -
```

kubectl logs -n load-balancer-demo -l app=web-app -f

```
10.42.0.1 - - [16/Jan/2026 12:38:41] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:47] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:49] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:52] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:57] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:38:59] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [16/Jan/2026 12:39:02] "GET /health HTTP/1.1" 200 -
```

7.2 – Ver estadísticas

kubectl top pods -n load-balancer-demo

```
NAME                       CPU(cores)   MEMORY(bytes)
web-app-65967466dd-2m4bg   1m           24Mi
web-app-65967466dd-5wwqc   2m           24Mi
web-app-65967466dd-8c6ws   1m           24Mi
web-app-65967466dd-fjb6s   2m           24Mi
web-app-65967466dd-tvgvb   2m           24Mi
```

kubectl top nodes

```
hect@A6Alumno13:~$ kubectl top nodes
NAME          CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
a6alumno14    181m         1%       1384Mi          17%
```

kubectl get events -n load-balancer-demo

```
hect@A6Alumno13:~$ kubectl get events -n load-balancer-demo
LAST SEEN   TYPE     REASON              OBJECT                                  MESSAGE
24m         Normal   Scheduled           pod/web-app-65967466dd-fjb6s            Successfully
p-65967466dd-fjb6s to a6alumno13
24m         Normal   Pulled              pod/web-app-65967466dd-fjb6s            Container ima
ent on machine
24m         Normal   Created             pod/web-app-65967466dd-fjb6s            Created cont
24m         Normal   Started             pod/web-app-65967466dd-fjb6s            Started cont
24m         Normal   Scheduled           pod/web-app-65967466dd-tvgvb            Successfully
p-65967466dd-tvgvb to a6alumno13
24m         Normal   Pulled              pod/web-app-65967466dd-tvgvb            Container ima
ent on machine
24m         Normal   Created             pod/web-app-65967466dd-tvgvb            Created cont
24m         Normal   Started             pod/web-app-65967466dd-tvgvb            Started cont
24m         Normal   SuccessfulCreate    replicaset/web-app-65967466dd           Created pod:
24m         Normal   SuccessfulCreate    replicaset/web-app-65967466dd           Created pod:
24m         Normal   ScalingReplicaSet   deployment/web-app                      Scaled up rep
to 5
```

7.3 – Ver detalles del deployment

kubectl describe deployment web-app -n load-balancer-demo

```
Name:                   web-app
Namespace:              load-balancer-demo
CreationTimestamp:      Fri, 16 Jan 2026 11:02:01 +0100
Labels:                 app=web-app
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=web-app
Replicas:               5 desired | 5 updated | 5 total | 5 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
```

kubectl rollout history deployment web-app -n load-balancer-demo

```
hect@A6Alumno13:~$ kubectl rollout history deployment web-app -n load-balancer-demo
deployment.apps/web-app
REVISION  CHANGE-CAUSE
1         <none>
```

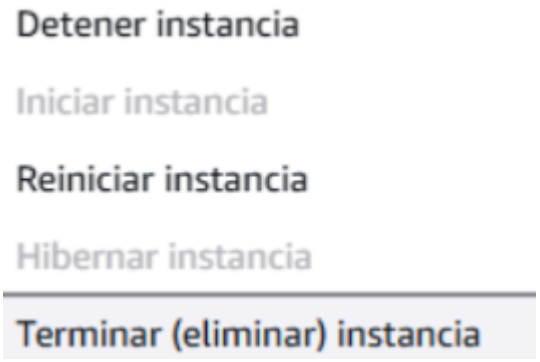
PARTE 8: ELIMINAR RECURSOS

8.1 – Limpiar Kubernetes

kubectl delete namespace load-balancer-demo

```
hect@A6Alumno13:~$ kubectl delete namespace load-balancer-demo
namespace "load-balancer-demo" deleted
```

8.2 – Eliminar instancia EC2



8.3 – Detener k3s

sudo systemctl stop k3s

```
hect@A6Alumno13:~$ sudo systemctl stop k3s
[sudo] password for hect:
```

sudo /usr/local/bin/k3s-uninstall.sh

```
+ id -u
+ [ 0 -eq 0 ]
+ K3S_DATA_DIR=/var/lib/rancher/k3s
+ /usr/local/bin/k3s-killall.sh
+ [ -s /etc/systemd/system/k3s.service ]
+ basename /etc/systemd/system/k3s.service
+ systemctl stop k3s.service
+ [ -x /etc/init.d/k3s* ]
```